BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC CMC
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TRIỂN KHAI PHẦN MỀM

WorkSphere

Xây dựng nền tảng quản lý yêu cầu hỗ trợ và công việc nội bộ doanh nghiệp

Bản chia sẻ công khai

Nhóm sinh viên lớp 24IT1

Thành viên 1

Thành viên 2

Thành viên 3

Thành viên 4

Hà Nội, tháng 11 năm 2026

Báo cáo buổi trình bày ngày 11/11/2026

Bảng phân công công việc

TT	Mã SV	Họ tên	Nội dung công việc	Điểm /10	Ghi chú
1		Thành viên 1			
2		Thành viên 2			
3		Thành viên 3			
4		Thành viên 4			

Điểm tự đánh giá 30%. Giảng viên 70%.

Mã sinh viên, phần việc và điểm tự đánh giá do nhóm xác nhận trước khi nộp.

Tóm tắt buổi trình bày

Nội dung	Cập nhật của nhóm
Đã làm được gì	Đóng gói web và API bằng Docker, chạy trên Compose với PostgreSQL, thiết lập Jenkins và SonarQube, giám sát API bằng Prometheus và Grafana. Đã thử sao lưu, khôi phục và quay về image trước.
Đã làm đến đâu	Nghiệm thu trên máy cá nhân ngày 09/11/2026. Jenkins #5 đạt 407 ca backend và 8 ca web, quality gate OK, triển khai và smoke đạt. #2 và #6 xác nhận Jenkins chặn deploy khi kiểm thử hoặc quality gate lỗi.
Cần hỏi thầy	Nhóm xin xác nhận phạm vi chấm phần local, cách demo khi tài nguyên máy có hạn, và điều kiện thực hiện phần cộng điểm SSH lên máy chủ công khai. Câu hỏi cụ thể ở mục 6.4.

Demo chính: trình bày Jenkins #5, mở web Docker cổng 18210, theo dõi yêu cầu hỗ trợ từ lúc gửi tới lúc đóng, sau đó mở dashboard giám sát.

MỤC LỤC

Bảng phân công và tóm tắt buổi trình bày....1
1. Giới thiệu chung....4
1.3. Phạm vi và kịch bản nghiệp vụ....5
2. Mô hình triển khai....6
2.1. Kiến trúc triển khai trên máy cá nhân....6
2.2. Luồng CI/CD từ GitHub đến Docker Compose....7
2.3. Ranh giới mạng và dữ liệu....8
3. Thiết lập hệ thống CI/CD....9
3.1. Cài đặt Jenkins và agent Linux....9
3.2. Jenkinsfile và báo cáo kiểm thử....10
3.3. Tích hợp SonarQube....11
3.4. Đóng gói và khởi động Docker Compose....12
3.5. Đối chiếu các lượt Jenkins và smoke....13
3.6. Quy trình phát hành và xử lý sự cố....14
4. Tự động deploy lên server qua SSH....15
5. Giám sát bằng Prometheus và Grafana....16
5.1. Cấu hình giám sát....16
5.2. Đọc dashboard và đối chiếu số liệu....17
5.3. Kiểm tra mất kết nối và phục hồi....18
6. Kết luận và cải tiến....19
6.4. Kịch bản demo và câu hỏi xin ý kiến....21
Phụ lục minh chứng....22

DANH SÁCH TỪ VÀ KÝ HIỆU VIẾT TẮT

Ký hiệu	Nội dung
CI/CD	Tích hợp và triển khai liên tục
API	Giao diện lập trình ứng dụng
JDK/JRE	Bộ công cụ phát triển và môi trường chạy Java
SHA	Mã định danh commit dùng để ghim phiên bản
JVM	Máy ảo Java
p95	Ngưỡng mà 95% mẫu độ trễ nằm dưới hoặc bằng
SSH	Giao thức truy cập máy chủ từ xa

DANH SÁCH HÌNH VẼ

Hình 1. Mô hình triển khai WorkSphere trên Docker Desktop
Hình 2. Luồng kiểm thử và triển khai từ GitHub đến Docker Compose
Hình 3. Jenkins #5 triển khai qua Compose và smoke PASS
Hình 4. Jenkins #6 nhận gate ERROR và bỏ qua stage triển khai
Hình 5. SonarQube hiển thị quality gate đạt của project triển khai
Hình 6. Bản WorkSphere Docker hiển thị yêu cầu đã đóng và tệp đính kèm
Hình 7. Prometheus ghi target worksphere-api ở trạng thái UP
Hình 8. Grafana hiển thị lưu lượng, độ trễ và bộ nhớ JVM của WorkSphere

1. Giới thiệu chung

1.1. Mục tiêu học phần

Đồ án tổ chức một quy trình triển khai có thể chạy lại từ Git. Jenkins lấy đúng phiên bản, kiểm thử tự động, phân tích chất lượng và đóng gói. Sau khi triển khai, nhóm kiểm tra luồng nghiệp vụ và theo dõi tình trạng API bằng số liệu thực.

1.2. Phần mềm được chọn

WorkSphere quản lý yêu cầu hỗ trợ và công việc nội bộ. Nhân viên gửi yêu cầu, trưởng phòng điều phối, người được giao xử lý và người gửi xác nhận kết quả. Web và ứng dụng Android dùng chung API Spring Boot. Môn này triển khai web, API và PostgreSQL trên máy cá nhân.

Thành phần	Phiên bản hoặc vai trò
Backend	Java 21, Spring Boot 4.1.1, Maven Wrapper và Flyway
Web	JavaScript tĩnh, Nginx reverse proxy
Dữ liệu	PostgreSQL 15, volumes riêng cho DB, uploads và mailbox
Môi trường	Windows, Docker Desktop Linux và Node.js 24

Repo worksphere-app chứa source và Dockerfile. Repo worksphere-deployment giữ pipeline, Compose, monitoring và hướng dẫn. Commit app dùng trong đợt triển khai:

4273f1ed907eb49ae0023ab1cacff7c25de8a691

Phạm vi nghiệm thu của học phần là bản web trong Docker ở cổng 18210. Ứng dụng Android và bản web thực hành ở cổng 18110 thuộc môi trường phát triển riêng.

1.3. Phạm vi và kịch bản nghiệp vụ

Tình huống demo là nhân viên Kinh doanh cần hỗ trợ máy tính. Minh gửi yêu cầu có ảnh đính kèm. Hùng, trưởng phòng IT, phân công cho Hải. Hải nhận việc, cập nhật kết quả xử lý và trả lại yêu cầu cho Minh xác nhận. Chuỗi này đi qua cả web, API, cơ sở dữ liệu, lưu trữ tệp và kiểm tra phân quyền.

Vai trò	Thao tác chính trong demo	Điều kiện quan sát
Người gửi	Đăng nhập, chọn loại hỗ trợ IT và đính kèm PNG.	Yêu cầu xuất hiện ở trạng thái OPEN.
Trưởng phòng IT	Mở hàng đợi của phòng và phân công người xử lý.	Lịch sử ghi người giao và người được giao.
Nhân viên IT	Nhận việc và ghi kết quả xử lý.	Yêu cầu chuyển IN_PROGRESS rồi RESOLVED.
Người gửi	Xem kết quả, tải tệp và xác nhận.	Yêu cầu CLOSED và tệp tải xuống khớp tệp gốc.

Nhóm chọn luồng này làm smoke test vì một lỗi ở reverse proxy, kết nối PostgreSQL, lưu trữ hoặc phân quyền đều có thể làm kịch bản thất bại dù trang đăng nhập đã mở. Bài kiểm thử gửi một thao tác trái quyền để xác nhận backend thực thi giới hạn của từng vai trò.

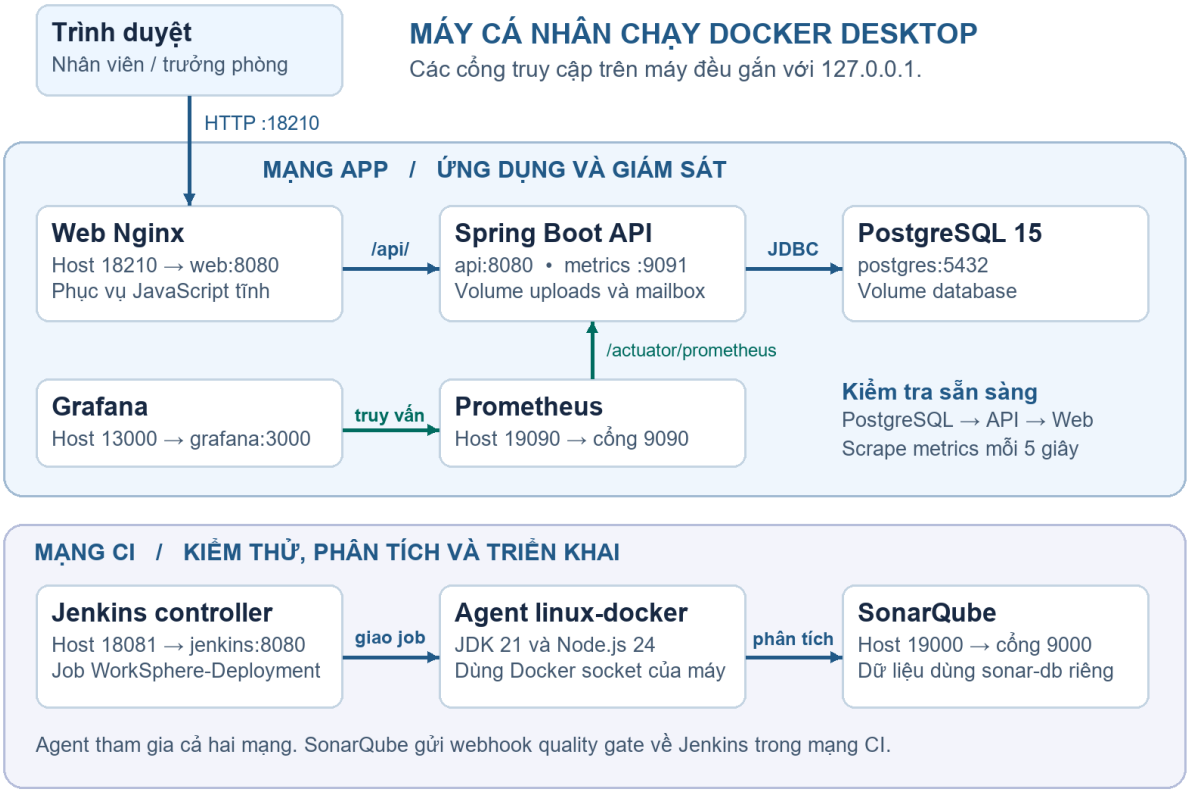
Bản triển khai sử dụng dữ liệu thực hành của công ty An Phát. Jenkins chạy bộ kiểm thử trên môi trường tách khỏi cơ sở dữ liệu demo. Các kết quả 407 ca backend và 8 ca web trong báo cáo thuộc lượt nghiệm thu Docker, phục vụ đúng phạm vi môn Triển khai phần mềm.

Hai kho mã có nhiệm vụ khác nhau. Khi cần sửa hành vi nghiệp vụ, nhóm thay đổi worksphere-app và kiểm thử tại đó. Khi cần đổi cách build, chạy hoặc theo dõi, nhóm cập nhật worksphere-deployment. Tệp versions.env nối hai kho bằng mã commit cụ thể.

2. Mô hình triển khai

2.1. Kiến trúc triển khai trên máy cá nhân

Nhóm dùng ba cấu hình Compose trong cùng project worksphere-deployment. Mạng app nối web, API, PostgreSQL và giám sát. Mạng ci nối Jenkins, agent và SonarQube. Agent tham gia cả hai mạng để điều khiển Docker và kiểm thử qua web sau triển khai.



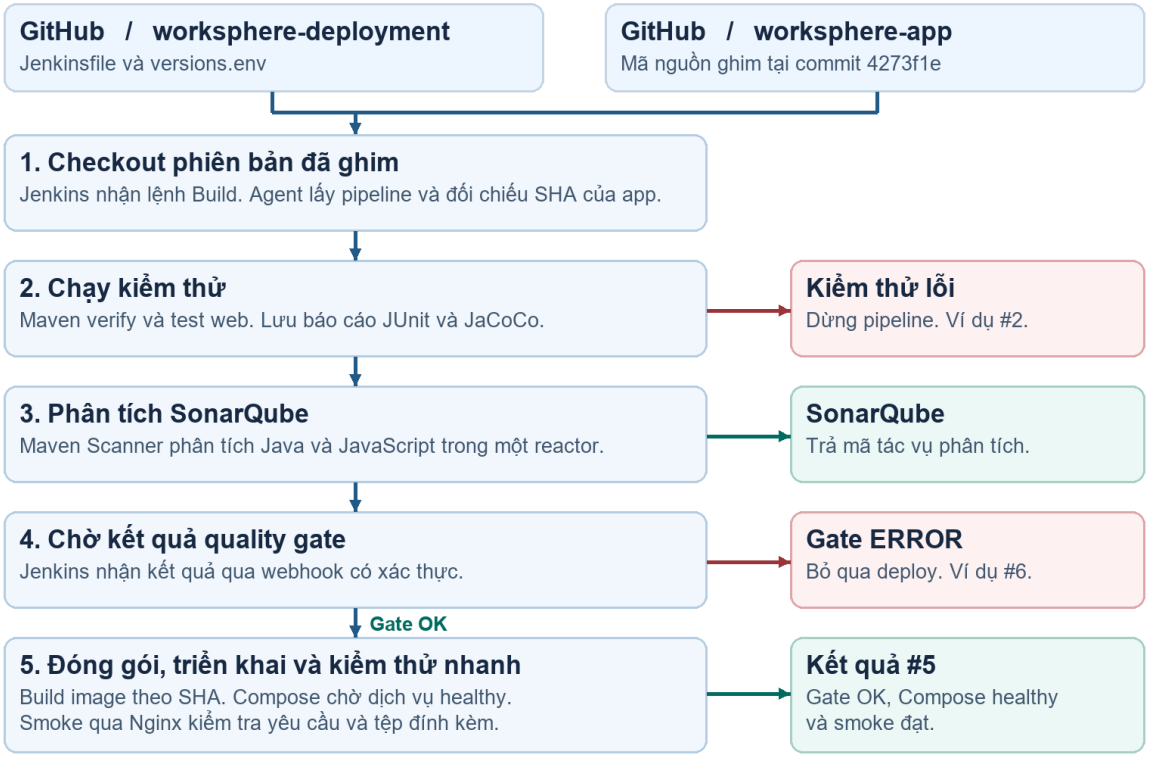
Hình 1. Mô hình triển khai WorkSphere trên Docker Desktop

Dịch vụ	Truy cập	Vai trò
Web	127.0.0.1:18210	Bản demo chạy trong Docker
Jenkins	127.0.0.1:18081	Controller và job trên máy cá nhân
SonarQube	127.0.0.1:19000	Phân tích Java, JavaScript và quality gate
Prometheus	127.0.0.1:19090	Thu metrics API theo chu kỳ 5 giây
Grafana	127.0.0.1:13000	Dashboard WorkSphere API
API và PostgreSQL	Mạng container	API 8080, metrics 9091 và PostgreSQL 5432

Nginx chuyển /api/ tới Spring Boot. Compose đợi PostgreSQL và API sẵn sàng trước khi mở web. Database, uploads và mailbox dùng volume riêng để giữ dữ liệu qua các lần tái tạo container.

2.2. Luồng CI/CD từ GitHub đến Docker Compose

Jenkins lấy Jenkinsfile từ kho triển khai. Agent đọc versions.env, checkout đúng commit của worksphere-app và đối chiếu SHA trước khi chạy test. Hình 2 thể hiện năm stage và các nhánh dừng đã kiểm chứng.



Hình 2. Luồng kiểm thử và triển khai từ GitHub đến Docker Compose

1. Tests chạy Maven verify trên JDK 21 và kiểm thử web trên Node.js 24. Backend dùng PostgreSQL thử riêng. Jenkins lưu kết quả JUnit và báo cáo độ phủ JaCoCo.

2. Maven Scanner gửi phân tích Java và JavaScript tới SonarQube. Jenkins chờ quality gate qua webhook. Lỗi test dừng các stage tiếp theo. Gate ERROR bỏ qua bước triển khai.

3. Gate OK mở stage đóng gói hai image theo SHA và chạy Compose. Khi các dịch vụ báo healthy, smoke đi qua Nginx để kiểm tra phân quyền, luồng yêu cầu hỗ trợ và tệp đính kèm. Sau khi triển khai, nhóm dùng Prometheus và Grafana để xác nhận API có số liệu.

Lượt Jenkins #5 dùng cấu hình triển khai tại commit dd20a3d. Mã image API và web đã nghiệm thu được lưu trong target.json của kho triển khai.

2.3. Ranh giới mạng và dữ liệu

Cả ba tệp Compose dùng tên project worksphere-deployment. Tên dịch vụ Docker như api, postgres và sonarqube được phân giải trong mạng phù hợp. Trình duyệt truy cập cổng gắn vào 127.0.0.1 của máy. PostgreSQL, API nội bộ và cổng metrics chỉ được truy cập qua mạng container.

Tệp cấu hình	Thành phần	Dữ liệu lưu qua các lần chạy
compose/compose.yaml	Nginx, Spring Boot, PostgreSQL 15 trong mạng app.	database, uploads, mailbox
jenkins/compose.yaml	Jenkins, agent, SonarQube và PostgreSQL của SonarQube. Agent vào cả mạng app và ci.	jenkins-home, agent-home, maven-home, dữ liệu SonarQube
monitoring/compose.yaml	Prometheus và Grafana trong mạng app.	prometheus-data, grafana-data

Luồng HTTP bắt đầu tại web:8080 phía sau cổng 18210. Nginx phục vụ tài nguyên tĩnh và chuyển đường dẫn /api/ tới api:8080. Nginx phân giải lại địa chỉ qua DNS Docker khi container API được tạo lại. Spring Boot dùng JDBC tới postgres:5432 và ghi tệp vào uploads.

Luồng giám sát dùng cổng quản trị 9091 của API. Prometheus kết nối trực tiếp tới api:9091 trên mạng app. Grafana truy vấn Prometheus qua mạng Docker. Đường đi này độc lập với việc nhân viên mở trang WorkSphere trên cổng 18210.

Compose chờ PostgreSQL báo healthy, sau đó chờ readiness của API rồi mới khởi động web. Để đối chiếu trạng thái toàn bộ luồng, nhóm kết hợp healthcheck với smoke qua Nginx. Dữ liệu thử được cấp bằng profile deployment-demo trên database dành riêng.

Mật khẩu và khóa của từng máy nằm trong file cấu hình local hoặc Jenkins Credentials. Khi khởi động lại, nhóm giữ các file cấu hình và volume tương ứng để cơ sở dữ liệu, lịch sử Jenkins và dashboard tiếp tục sử dụng được.

3. Thiết lập hệ thống CI/CD

3.1. Cài đặt Jenkins và agent Linux

Nhóm chạy controller Jenkins 2.580.2 trong Docker Desktop và dành công việc build cho agent linux-docker. Controller giữ lịch sử, cấu hình và kết quả kiểm thử trong jenkins-home. Agent có JDK 21, Node.js 24 và Docker CLI. Cache Maven đặt trong volume riêng để các lần chạy sau tái sử dụng thư viện.

Jenkins Configuration as Code tải cấu hình controller. Script cấu hình job WorkSphere-Deployment trỏ tới Jenkinsfile của kho triển khai. Agent kết nối inbound qua WebSocket, có một executor và dùng Docker socket của máy để khởi động PostgreSQL kiểm thử và triển khai Compose.

Thao tác	Cấu hình hoặc kết quả cần thấy
Chuẩn bị	Docker Desktop có Linux engine. Có Node.js 24, versions.env và cấu hình local cho app.
Tạo cấu hình CI	configure-ci.mjs prepare ghi biến local cho nhánh triển khai và quyền truy cập Docker socket.
Khởi động SonarQube	Khởi động sonarqube và sonar-db, chờ API trạng thái trả UP rồi cấu hình project.
Khởi động Jenkins	Mở cổng 18081, kiểm tra job WorkSphere-Deployment đã xuất hiện.
Kết nối agent	configure-ci.mjs agent lấy thông tin kết nối. Khởi động agent và kiểm tra node linux-docker Online.

```
node scripts/configure-ci.mjs prepare docs/deployment-plan-20261105
$ci = @('compose','--env-file','.env.local','--env-file','.env.ci',
        '-f','jenkins/compose.yaml')
docker @ci up -d sonarqube
# Chờ SonarQube UP trước khi tiếp tục.
node scripts/configure-ci.mjs sonar
docker @ci up -d --build jenkins
# Chờ trang Jenkins tải được trước khi cấu hình agent.
node scripts/configure-ci.mjs agent
docker @ci up -d --build agent
```

Cấu hình và thứ tự chạy nằm trong jenkins/compose.yaml và runbooks/LOCAL.md. Nhóm kiểm tra node đã kết nối trước khi bấm Build để job nhận đúng môi trường JDK 21.

3.2. Jenkinsfile và báo cáo kiểm thử

Job chạy trên label linux-docker và tắt các lượt build đồng thời. Mỗi lượt làm sạch workspace, lấy đúng phiên bản cấu hình triển khai và lấy mã ứng dụng ở APP_COMMIT. Jenkins dừng ở bước checkout nếu mã thực tế khác mã đã ghim.

Stage	Điều kiện chuyển tiếp	Đầu ra đối chiếu
Checkout pinned source	Workspace sạch, SHA app khớp versions.env.	Mã commit cấu hình, mã app và phiên bản công cụ.
Tests	Các ca web và Maven verify đều đạt.	Log Node.js, XML JUnit và JaCoCo.
SonarQube analysis	SonarQube sẵn sàng, Maven Scanner gửi phân tích thành công.	Mã tác vụ phân tích và report-task.txt.
Quality gate	Webhook trả gate OK trong thời gian chờ.	Trạng thái quality gate trong log Jenkins.
Build and deploy with Compose	Docker build, healthcheck và smoke đều thành công.	Image ID, deployment.txt, smoke.json và danh sách container.

Stage Tests chạy các ca web về phiên đăng nhập và lịch sử lượt quét. Trước Maven verify, ci-test-backend.sh tạo PostgreSQL thử trên mạng CI cho các lớp kiểm thử nhận JDBC qua biến môi trường. Những lớp dùng Testcontainers tự tạo PostgreSQL khi chạy. Trong 407 ca backend của lượt #5 có 51 ca Workplace và 8 ca Authorization.

Fixture QR được sửa để đồng hồ test và hạn của mã dùng cùng mốc. Việc này giúp kiểm tra ổn định trường hợp mã hợp lệ, hết hạn, phát lại và trạm đóng. Mốc mới được kiểm chứng trong lần chạy backend đầy đủ.

Khối post của Jenkins công bố JUnit và lưu các tệp kết quả kể cả khi pipeline thất bại. Nhóm dùng Console Output xác định stage dừng và dùng test report để đối chiếu số ca. Cấu hình triển khai dùng JDK 21 và không phụ thuộc IDE trên máy.

3.3. Tích hợp SonarQube

SonarQube chạy với PostgreSQL riêng trong mạng ci. Jenkins truy cập SonarQube qua địa chỉ sonarqube:9000. Trang quản trị mở ở cổng 19000 trên máy cá nhân. Maven Scanner phân tích Java và JavaScript theo jenkins/sonar-reactor.xml và đọc JaCoCo XML từ lượt kiểm thử backend.

Bước	Cách thực hiện trong WorkSphere
Cấu hình project	configure-ci.mjs sonar tạo project triển khai và project thử từ chối, đồng thời cài webhook gửi về Jenkins.
Bắt đầu phân tích	withSonarQubeEnv cấp kết nối SonarQube và token từ Jenkins Credentials. Jenkins lấy mã tác vụ từ scanner.
Chờ kết quả	waitForQualityGate nhận kết quả qua webhook nội bộ có secret. Bước chờ gate có thời hạn 12 phút.
Quyết định	Gate OK mới chuyển tới build image. Gate ERROR kết thúc lượt build và bỏ qua stage triển khai.

Project worksphere-deployment dùng để đánh giá phiên bản triển khai. Lượt #5 trả gate OK. Lượt #6 chuyển phân tích sang worksphere-rejection-check với điều kiện độ phủ 100% để chứng minh Jenkins chặn triển khai khi gate ERROR. Kết quả #6 là ca thử từ chối có chủ đích.

Nhóm đối chiếu màn Overview của project với Console Output. Ảnh SonarQube ngày 10/11 hiện Passed cho project WorkSphere. Ảnh Jenkins #6 ghi rõ stage Compose bị bỏ qua. Hai ảnh thể hiện trạng thái của hai project và hai mục tiêu kiểm tra khác nhau.

Lượt #4 cho thấy lỗi cấu hình scanner có thể xảy ra dù toàn bộ 407 ca backend đã đạt. Đường dẫn phân tích bị ghép lặp thư mục backend. Nhóm sửa bằng reactor Maven chung cho Java và web, sau đó #5 xác nhận phân tích, gate và triển khai đều đạt.

3.4. Đóng gói và khởi động Docker Compose

Dockerfile backend dùng hai giai đoạn. Giai đoạn đầu dùng Temurin JDK 21 và Maven Wrapper để tạo JAR. Giai đoạn chạy lấy JAR sang Temurin JRE 21 Alpine và chạy bằng tài khoản worksphere. Pipeline đã chạy kiểm thử trước khi đóng gói.

Dockerfile web dùng Nginx 1.28 Alpine, sao chép JavaScript, CSS, HTML và nginx.conf. Hai image được gắn tag theo APP_COMMIT. Compose chạy chúng cùng PostgreSQL 15, gắn volume database, uploads và mailbox, rồi chờ healthcheck.

Dịch vụ	Kiểm tra sẵn sàng	Vị trí hoặc cổng
PostgreSQL	pg_isready xác nhận máy chủ nhận kết nối.	postgres:5432
API	Actuator readiness trả thành công trong container API.	api:9091/actuator/health/readiness
Web	Nginx trả trang chủ sau khi API healthy.	127.0.0.1:18210

Các lệnh sau chạy từ thư mục kho triển khai đã checkout app đúng commit và tạo .env.local theo runbook.

```
$app = @('compose','--env-file','versions.env','--env-file','.env.local',
         '-f','compose/compose.yaml')
docker @app config --quiet
docker @app up -d --build --wait --wait-timeout 180
docker @app ps
New-Item -ItemType Directory -Path artifacts -Force | Out-Null
node --env-file=.env.local scripts/smoke.mjs workflow artifacts/smoke.json
```

Tệp target.json ghi hai image ID local đã dùng khi nghiệm thu. Khi đối chiếu với Jenkins #5, nhóm dùng cả mã commit, tag image và image ID. Registry digest chỉ được ghi nhận sau khi đẩy image lên registry.

3.5. Đối chiếu các lượt Jenkins và smoke

Nhóm sử dụng ba lượt chạy để kiểm tra quyết định triển khai. Một lượt đi hết pipeline và hai lượt xác nhận các điều kiện chặn hoạt động ở những giai đoạn khác nhau.

Hạng mục kiểm tra	Kết quả ghi nhận
Backend P0	407 đạt, 0 thất bại, 0 lỗi, 0 bỏ qua trên JDK 21
Web P0	8/8 ca đạt trên Node.js 24
Jenkins test cố ý fail	Build #2 thất bại, stage triển khai bị bỏ qua
Jenkins đầy đủ	Jenkins #5 đạt 407 test backend, 8 test web, quality gate OK, Compose healthy và smoke đạt.
Gate từ chối	Jenkins #6 đạt 407 test nhưng SonarQube trả ERROR. Jenkins bỏ qua deploy và kết thúc FAILURE đúng kỳ vọng.
Compose và smoke	Ba dịch vụ healthy. Luồng hỗ trợ đi tới CLOSED. PNG tải lại đúng từng byte.

Luồng smoke mở phiên cho các vai trò thử, tạo yêu cầu có PNG, kiểm tra thao tác trái quyền rồi phân công cho nhân viên IT. Sau khi xử lý và xác nhận, script kiểm tra trạng thái CLOSED, tải lại tệp để so sánh từng byte và xác nhận phiên đã đăng xuất bị thu hồi.

Ngày 10/11, nhóm bật lại các dịch vụ và chạy mới luồng smoke. Ảnh WorkSphere trong phụ lục cho thấy yêu cầu đã đóng, lịch sử xử lý và tệp đính kèm. Ảnh Jenkins #5 và #6 chụp từ lịch sử ngày 09/11.

3.6. Quy trình phát hành và xử lý sự cố

Khi phát hành phiên bản mới, nhóm chốt một commit app đã review, cập nhật APP_COMMIT trong versions.env và lưu thay đổi trên nhánh triển khai. Trên Jenkins, chọn Build with Parameters và để hai cờ minh họa lỗi ở trạng thái tắt. Sau khi job hoàn thành, lưu mã build, commit, image ID và báo cáo smoke.

Hiện tượng đã gặp	Cách xác định và xử lý
Jenkins #1 dừng khi ghi cache Maven	Đọc lỗi ghi volume trong agent. Sửa quyền sở hữu và cấu hình image agent trước khi thử lại.
Jenkins #3 bị gián đoạn	Docker Desktop mất phản hồi khi chạy đồng thời nhiều dịch vụ. Khôi phục engine, chạy theo chặng và kiểm tra lại container.
Jenkins #4 dừng ở Sonar	Backend đã đạt. Kiểm tra đường dẫn scanner, sửa reactor và nghiệm thu lại bằng #5.
Web mở nhưng nghiệp vụ lỗi	Kiểm tra health API và PostgreSQL, sau đó chạy smoke qua Nginx để phân biệt lỗi giao diện, proxy và dữ liệu.
Grafana chưa có đồ thị	Kiểm tra target Prometheus và khoảng thời gian truy vấn. Chạy smoke để có lưu lượng rồi đợi chu kỳ scrape.

Trong buổi trình bày, nhóm có thể mở kết quả #5 để chỉ ra đường đi thành công, dùng #2 và #6 để giải thích các trường hợp dừng rồi thao tác một yêu cầu trên web. Khi cần chạy pipeline trực tiếp, nhóm khởi động SonarQube đúng thời điểm và theo dõi bộ nhớ Docker.

Để dừng demo, dùng Compose stop theo nhóm dịch vụ như runbook. Tạo lại container sẽ dùng các volume hiện có. Khi xử lý lỗi dữ liệu hoặc thử quay lui, nhóm dùng project phục hồi riêng trước khi thay đổi môi trường đang trình bày.

4. Tự động deploy lên server qua SSH

4.1. Tình trạng thực hiện

Nhóm đã nghiệm thu pipeline đến Docker Compose trên máy cá nhân. Triển khai tự động qua SSH lên máy chủ công khai thuộc phần cộng điểm theo mẫu. Hạng mục này chờ máy chủ được phép sử dụng và điều kiện nghiệm thu của giảng viên.

4.2. Phương án triển khai khi có máy chủ

Bước dự kiến	Cách thực hiện	Kết quả cần lưu
Chuẩn bị máy chủ	Cài Docker và Compose trên Linux, chuẩn bị tên miền hoặc địa chỉ truy cập. Xác nhận HTTPS với giảng viên.	Máy chủ, cổng truy cập và phạm vi mở mạng.
Chuẩn bị bản phát hành	Sau quality gate OK, đẩy image API và web lên registry. Ghi digest và cặp phiên bản đã kiểm thử.	Mã commit và digest của từng image.
Kết nối qua SSH	Đặt khóa trong Jenkins Credentials, xác thực host key, dùng tài khoản triển khai được cấp.	Jenkins log xác nhận stage SSH chạy.
Khởi động từ xa	Cấp cấu hình môi trường trên server, chạy Compose với image đã duyệt, chờ healthcheck.	Danh sách container và trạng thái healthy.
Kiểm tra và phục hồi	Mở URL từ mạng ngoài, chạy smoke, xác nhận kế hoạch quay lui theo schema và bản sao lưu.	URL truy cập, smoke và mốc image để quay lui.

Trong phương án này, web là điểm truy cập của người dùng. PostgreSQL, SonarQube, Jenkins và cổng metrics được giữ trong phạm vi quản trị phù hợp. Phần giám sát của môn vẫn có thể trình bày trên máy cá nhân theo mẫu.

5. Giám sát bằng Prometheus và Grafana

5.1. Cấu hình giám sát

Spring Boot Actuator và Micrometer cung cấp số liệu qua cổng quản trị 9091. Prometheus thu số liệu định kỳ. Grafana dùng Prometheus làm datasource để trình bày trạng thái API, lưu lượng theo mã HTTP, số lỗi mỗi giây, độ trễ p95 và bộ nhớ JVM.

Thành phần	Cấu hình	Cách kiểm tra
Spring Boot API	Endpoint /actuator/prometheus trên cổng nội bộ 9091.	Trạng thái readiness trong container và số liệu scrape.
Prometheus	Job worksphere-api gọi api:9091 mỗi 5 giây, timeout 3 giây.	Trang Targets cổng 19090 hiển thị UP.
Grafana	Datasource và dashboard nạp từ thư mục monitoring/grafana/provisioning.	Dashboard WorkSphere API trên cổng 13000.

Tệp monitoring/compose.yaml đưa cả hai dịch vụ vào mạng app để Prometheus truy cập API bằng tên container. Dashboard được lưu trong Git và nạp khi chạy image Grafana. Dữ liệu Prometheus nằm trong volume prometheus-data.

```
$metrics = @('compose','--env-file','.env.ci',
             '-f','monitoring/compose.yaml')
docker @metrics up -d --build
docker @metrics ps
```

Nhóm mở Prometheus ở cổng 19090, xác nhận target worksphere-api UP, sau đó mở Grafana cổng 13000. Để dashboard có điểm dữ liệu, chạy luồng smoke qua cổng 18210 và chọn khoảng thời gian bao phủ lượt chạy.

5.2. Đọc dashboard và đối chiếu số liệu

Dashboard WorkSphere API có năm panel. Trạng thái UP phản ánh kết nối scrape giữa Prometheus và API. Các panel khác giúp phân biệt một API vẫn trả lời với một API đang có lỗi, tăng độ trễ hoặc dùng thêm bộ nhớ.

Panel	Cách đọc	Ý nghĩa khi chạy demo
API scrape availability	Chỉ số up của job worksphere-api.	UP khi truy xuất metrics thành công. DOWN khi Prometheus không thu được metrics.
HTTP request rate by status	Tốc độ yêu cầu tách theo mã HTTP, bỏ endpoint Actuator.	Thấy hoạt động khi chạy luồng hỗ trợ qua Nginx.
HTTP error rate	Tách tốc độ phản hồi 4xx và 5xx.	Ca sai quyền và phiên đã thu hồi có thể tạo 4xx theo dự kiến.
HTTP latency p95	Tính phân vị từ histogram thời gian phản hồi.	Cho biết ngưỡng phản hồi của phần lớn yêu cầu trong cửa sổ truy vấn.
JVM memory used	Bộ nhớ heap và non-heap theo JVM.	So sánh mức sử dụng trước và sau khi API khởi động lại.

Lượt chạy thử có 56 mẫu độ trễ trước khi dừng và 6 sau khi phục hồi. Các số đếm thuộc hai thời điểm của tiến trình. Khi API khởi động lại, bộ đếm có thể bắt đầu lại. Nhóm đối chiếu trạng thái, thời điểm và luồng smoke trước khi so sánh số liệu.

Trong ảnh Grafana ngày 10/11, trạng thái UP cùng các đường lưu lượng và bộ nhớ cho thấy datasource đang đọc dữ liệu. Các ảnh phục vụ kiểm chứng giám sát trong môi trường demo. Nhóm sẽ cần một kịch bản tải riêng nếu đánh giá thông lượng hoặc độ trễ ở quy mô lớn.

5.3. Kiểm tra mất kết nối và phục hồi

Nhóm kiểm tra tình huống API ngừng trong khi Prometheus và Grafana vẫn hoạt động. Bài thử giữ web và PostgreSQL để quan sát tác động của riêng API. Jenkins không chạy stage triển khai đồng thời với thao tác này.

```
docker @app stop api
# Mở Prometheus Targets và đợi trạng thái DOWN.
docker @app up -d --no-build --wait api
# Đợi target trở về UP rồi kiểm tra yêu cầu và tệp đã tạo.
node --env-file=.env.local scripts/smoke.mjs verify artifacts/smoke.json
```

Mốc	Giờ UTC+7	Trạng thái	Dữ liệu đọc qua Grafana
Trước khi dừng	11:55:27	UP	56 request, 10 mã lỗi, 56 mẫu độ trễ
API đã dừng	11:55:35	DOWN	Target tạm thời không scrape được
API phục hồi	11:55:59	UP	6 request, 2 mã lỗi, 6 mẫu độ trễ

Prometheus ghi UP → DOWN → UP. Grafana đọc metrics thật và smoke tải lại đúng tệp sau khi API phục hồi.

Sau khi API phục hồi, nhóm xác nhận hai việc. Target trở lại UP chứng minh đường scrape đã thông. Smoke verify xác nhận người dùng vẫn mở được yêu cầu CLOSED và tải đúng tệp, qua đó kiểm tra phần nghiệp vụ sau sự cố.

Các mã lỗi HTTP trong bảng bao gồm phản hồi từ ca kiểm thử chủ động. Nhóm dùng nhật ký theo thời điểm để phân biệt mã lỗi dự kiến với sự cố vận hành. Các mốc UP và DOWN là thời điểm quan sát trong lượt nghiệm thu ngày 09/11.

6. Kết luận và cải tiến

6.1. Sao lưu, khôi phục và quay lui phiên bản

Bản sao lưu gồm dữ liệu PostgreSQL, tệp uploads và manifest ghi commit app cùng image ID. Script dừng API trong lúc xuất dữ liệu để yêu cầu và tệp thuộc cùng một mốc. Sau khi sao lưu, script khởi động lại API.

Giai đoạn	Thao tác và kiểm tra
Sao lưu	Tạo một yêu cầu mẫu, ghi smoke.json, chạy pg_dump và nén uploads. Giữ bộ sao lưu trong thư mục riêng.
Khôi phục	Tạo project Compose riêng ở cổng 18211. Khôi phục database và uploads rồi bật web, API.
Kiểm tra	Chạy smoke verify với URL 18211. Đối chiếu trạng thái CLOSED và nội dung tệp.
Quay lui image	Thử image trước và image #5 trên bản khôi phục. Kiểm tra schema Flyway tương thích rồi chạy smoke cho từng lượt.

```
node scripts/backup-restore.mjs backup artifacts/backup-demo
node scripts/backup-restore.mjs restore artifacts/backup-demo `
  worksphere-restore-demo 18211
$env:DEPLOY_URL = 'http://127.0.0.1:18211'
node --env-file=.env.local scripts/smoke.mjs verify artifacts/smoke.json
Remove-Item Env:DEPLOY_URL
```

Đã chạy lại image trước và trở về image #5 trên bản restore cổng 18211. Tệp và yêu cầu khớp ở cả hai lượt. Cả hai image dùng schema tương thích. Khi schema khác, phương án phục hồi là dùng bản sao lưu phù hợp và kiểm tra trong project riêng.

6.2. Kết quả và giới hạn

Đã nghiệm thu local từ checkout đến triển khai, chặn deploy khi test hoặc gate lỗi, giám sát và phục hồi. SSH public là phần cộng điểm chờ máy chủ.

Môi trường kiểm tra là máy cá nhân dùng Docker Desktop Linux. Bản triển khai đã có mã nguồn ghim, image local và bằng chứng quay lui cùng schema. Kết quả vận hành hiện gồm smoke và tình huống dừng API ngắn. Việc đo thời gian phục hồi, tải và triển khai công khai thuộc các bước tiếp theo.

6.3. Kế hoạch hoàn thiện

Hạng mục	Việc tiếp theo
Chuẩn bị nộp	Xác nhận MSSV, phần việc và điểm tự đánh giá. Chốt thứ tự thao tác khi trình bày.
Pipeline	Giảm áp lực bộ nhớ bằng cách chạy dịch vụ theo chặng khi demo. Sau buổi trình bày, cân nhắc lưu image theo mã phiên bản trong registry để dễ đưa sang máy khác.
Vận hành	Diễn tập lại trên máy trình bày. Khi có hạ tầng, đo thời gian khôi phục và bổ sung cảnh báo API mất kết nối.
Phần mềm	Cải thiện thông báo trên web khi API tạm ngừng và kiểm thử thao tác gửi lại sau khi kết nối phục hồi.

6.4. Kịch bản demo và câu hỏi xin ý kiến

Thứ tự trình bày dự kiến

Phần	Thao tác	Kết quả cần chỉ ra
1. Phiên bản	Mở kho triển khai và Jenkins #5.	Commit đã ghim, 407 ca backend và 8 ca web đạt.
2. Điều kiện deploy	Mở quality gate và đối chiếu với Jenkins #2 và #6.	Gate OK cho phép deploy. Test lỗi hoặc gate ERROR thì deploy bị bỏ qua.
3. Web	Mở bản Docker, gửi yêu cầu kèm tệp và minh họa phân công, xử lý, xác nhận.	Yêu cầu tới CLOSED. Tệp đính kèm tải lại đúng.
4. Giám sát	Mở Prometheus Targets và dashboard Grafana.	Target UP, biểu đồ có dữ liệu. Dùng kết quả UP → DOWN → UP đã lưu để giải thích ca mất kết nối và phục hồi.

Câu hỏi nhóm xin thầy góp ý

1. Với phần bắt buộc, nhóm đã nghiệm thu đầy đủ trên máy cá nhân. Thầy muốn nhóm bổ sung minh chứng hoặc thao tác nào trước khi nộp cuối kỳ?

2. Trong thời gian trình bày, nhóm có thể mở lượt Jenkins #5 đã lưu, chạy live luồng nghiệp vụ và mở dashboard, rồi dùng #2 và #6 để minh họa ca lỗi không? Nếu cần chạy lại cả pipeline trực tiếp, nhóm sẽ chuẩn bị thời gian và tài nguyên.

3. Nếu làm phần cộng điểm SSH, nhóm có thể dùng VPS riêng hay cần môi trường của trường? Khi nghiệm thu phần này, thầy yêu cầu URL public, HTTPS hoặc điều kiện nào khác?

Phụ lục minh chứng

Ảnh chụp ngày 10/11/2026 sau khi bật lại các dịch vụ trên máy cá nhân. Jenkins #5 và #6 là các lượt chạy đã lưu ngày 09/11. Luồng smoke trên web được chạy lại ngày 10/11, xác nhận yêu cầu đã đóng và tệp đính kèm khớp.

Mốc	Kết quả
Phiên bản app	4273f1ed907eb49ae0023ab1cacff7c25de8a691
Jenkins	Jenkins #5 đạt 407 test backend, 8 test web, quality gate OK, Compose healthy và smoke đạt.
Chặn deploy	Build #2 test lỗi có chủ ý. Jenkins #6 đạt 407 test nhưng SonarQube trả ERROR. Jenkins bỏ qua deploy và kết thúc FAILURE đúng kỳ vọng.
Phục hồi	Project riêng tại cổng 18211. Yêu cầu và ảnh đính kèm khớp.
Giám sát	Prometheus ghi UP → DOWN → UP. Grafana đọc metrics thật và smoke tải lại đúng tệp sau khi API phục hồi.

Tài liệu tham khảo

Mẫu báo cáo học phần Triển khai phần mềm, Trường Đại học CMC.

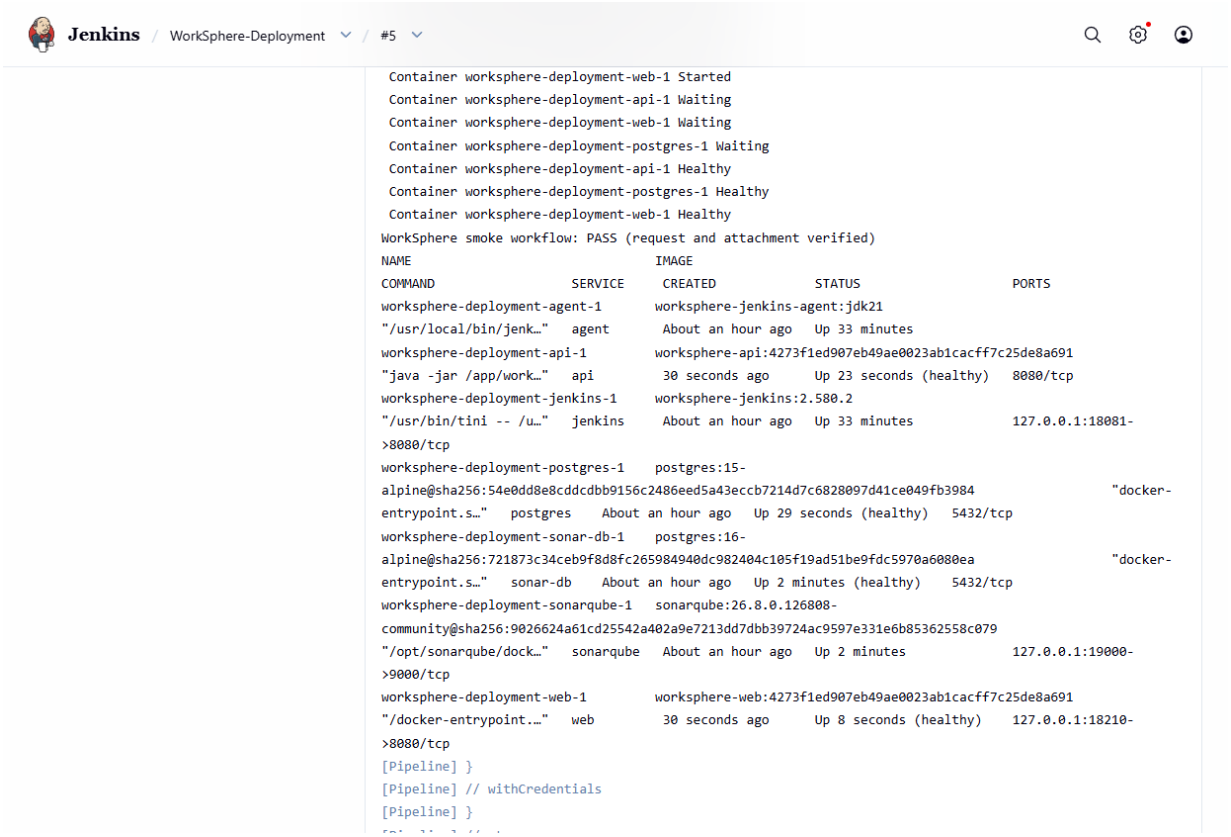
Kho triển khai — https://github.com/Khiemnh158/worksphere-deployment

Kho mã nguồn WorkSphere — https://github.com/Khiemnh158/worksphere-app

Trong kho triển khai, xem evidence/LOCAL_2026-11-09.md và runbooks/LOCAL.md.

Lượt triển khai thành công

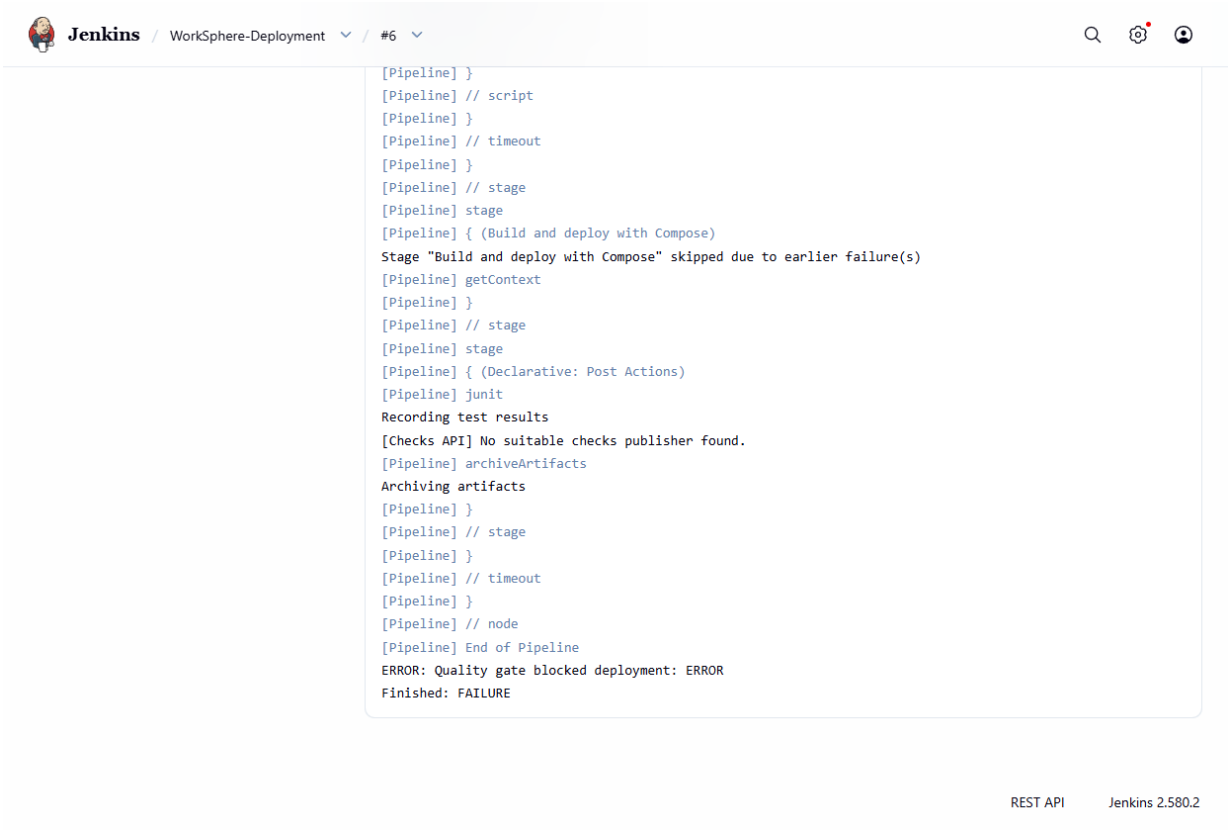
Trong Console Output của Jenkins #5, web, API và PostgreSQL đều báo healthy. Dòng WorkSphere smoke workflow PASS xác nhận luồng hỗ trợ và tệp đính kèm qua web. Ảnh cho thấy stage triển khai đã thực thi sau khi quality gate đạt.



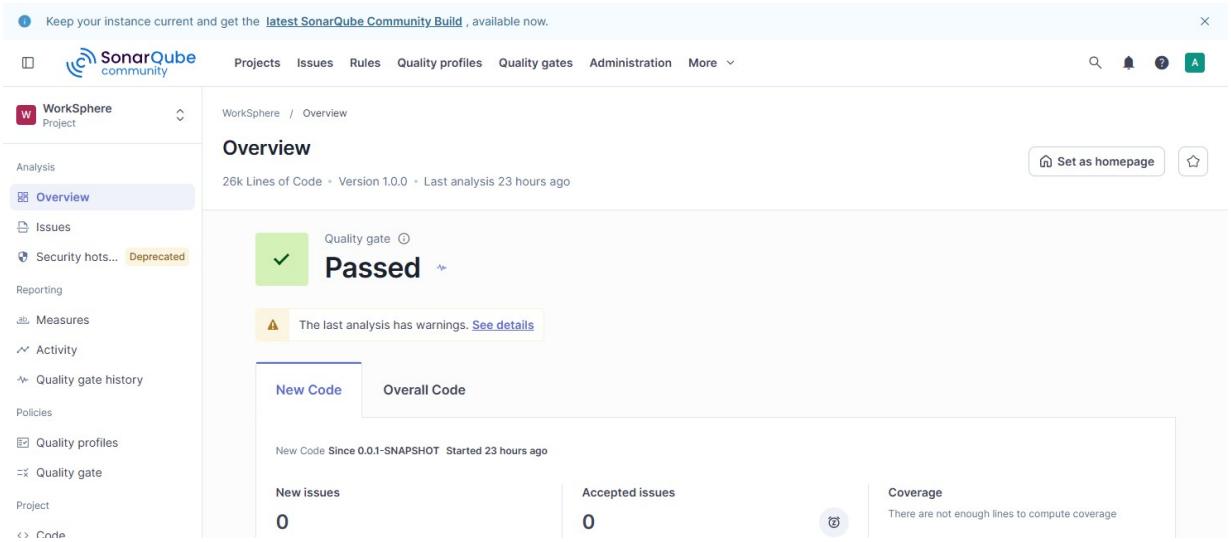
Hình 3. Jenkins #5 triển khai qua Compose và smoke PASS

Lượt triển khai bị chặn

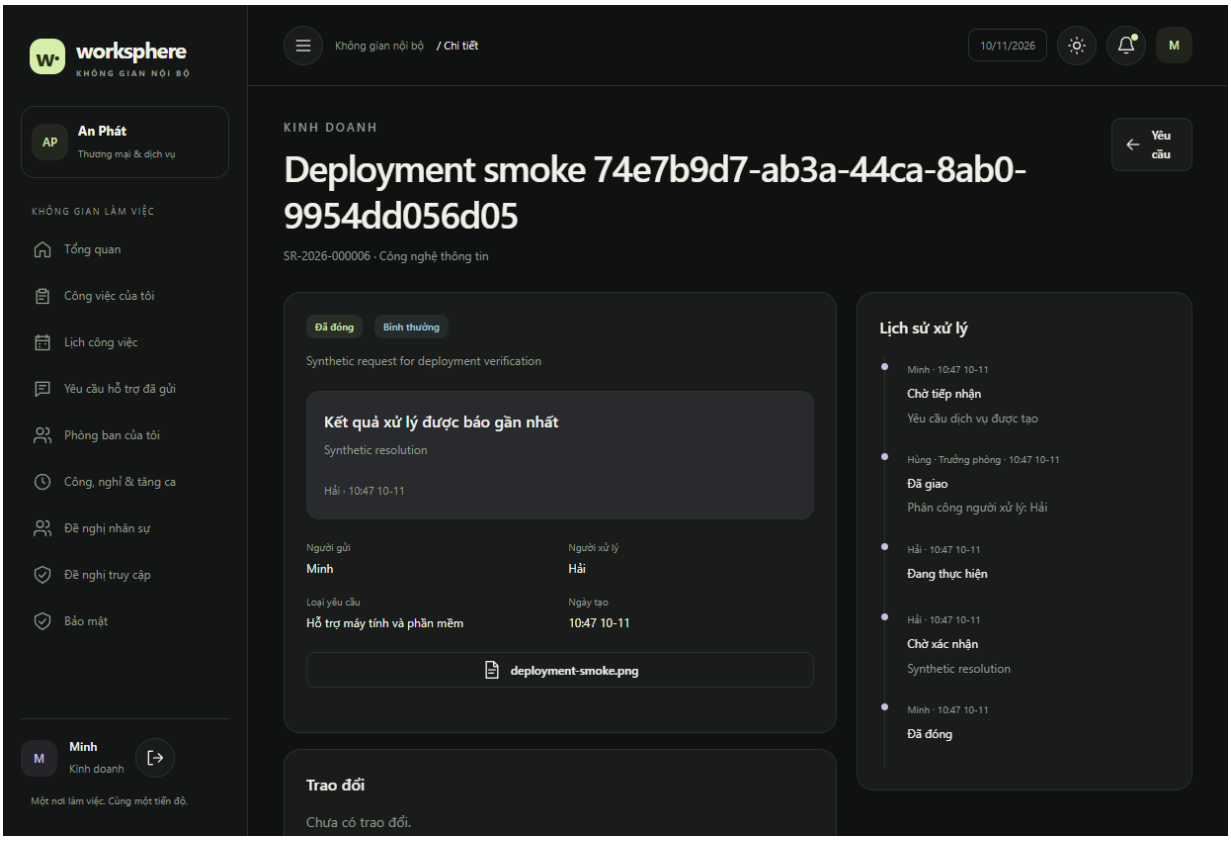
Jenkins #6 nhận ERROR từ project thử từ chối. Console Output ghi stage Build and deploy with Compose bị bỏ qua và kết thúc FAILURE. Jenkins #2 dùng cờ lỗi test có chủ đích để kiểm tra điều kiện dừng ở stage Tests.



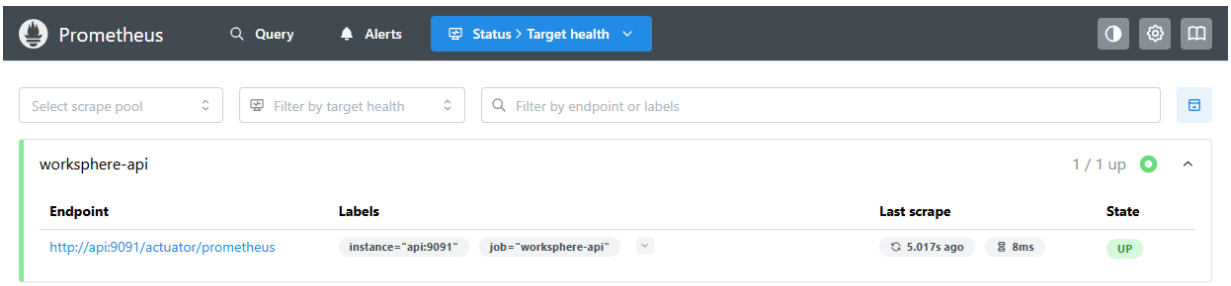
Hình 4. Jenkins #6 nhận gate ERROR và bỏ qua stage triển khai



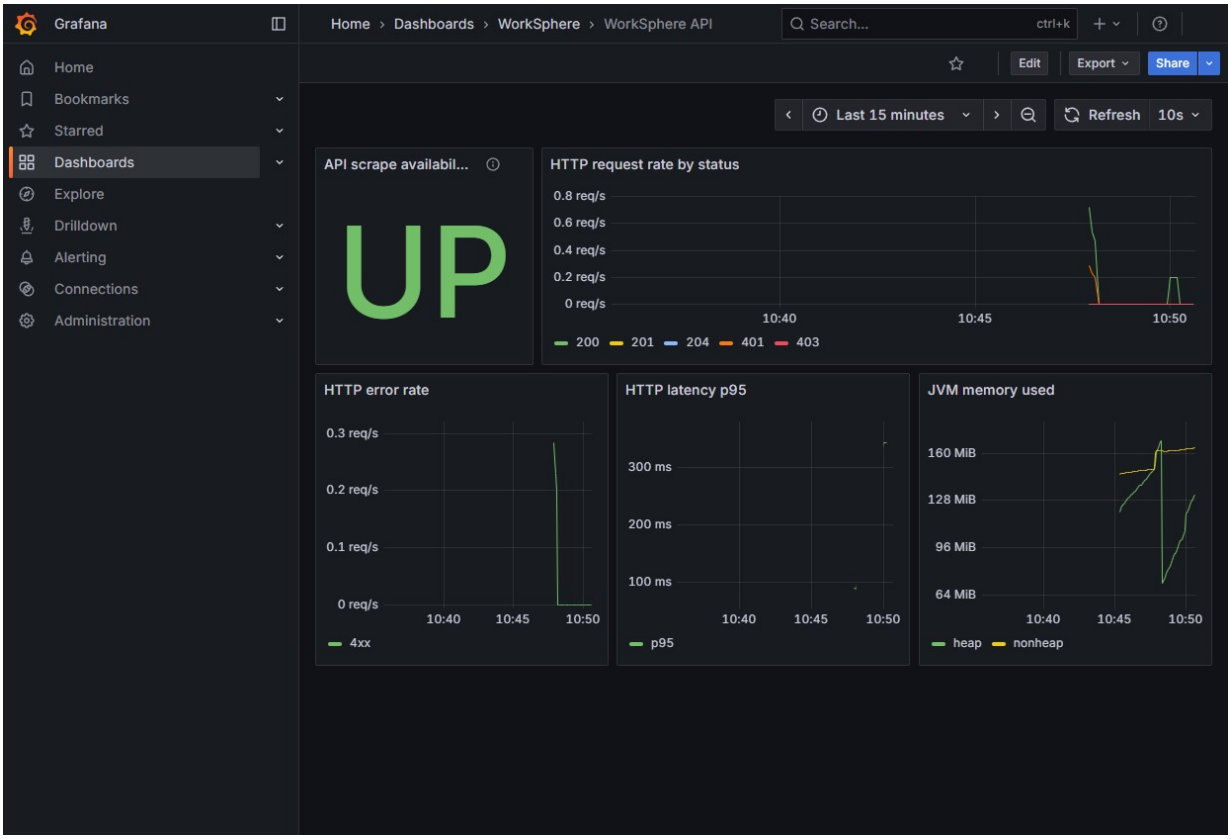
Hình 5. SonarQube hiển thị quality gate đạt của project triển khai



Hình 6. Bản WorkSphere Docker hiển thị yêu cầu đã đóng và tệp đính kèm



Hình 7. Prometheus ghi target worksphere-api ở trạng thái UP



Hình 8. Grafana hiển thị lưu lượng, độ trễ và bộ nhớ JVM của WorkSphere